Tarea para AD04.

Una empresa dispone de una base de datos contiene las tablas con la información necesaria para su gestión. Las tablas son las siguientes:

- Tabla **departamentos** que contiene información de los diferentes departamentos que tiene la empresa. La clave principal **dept_no**.
- Tabla **empleados** que contiene la información de los diferentes empleados que tiene la empresa. Tiene como clave principal **emp_no** y ajena **dept_no** que relaciona con la tabla **departamentos**
- La creación de las tablas y sus relaciones se dan en el fichero **TablasEmpleMYSQL_FK.sql** que se adjunta.

La base de datos que se utilizará será MySQL (utiliza hibernate como esquema y root-root como usuario y contraseña).

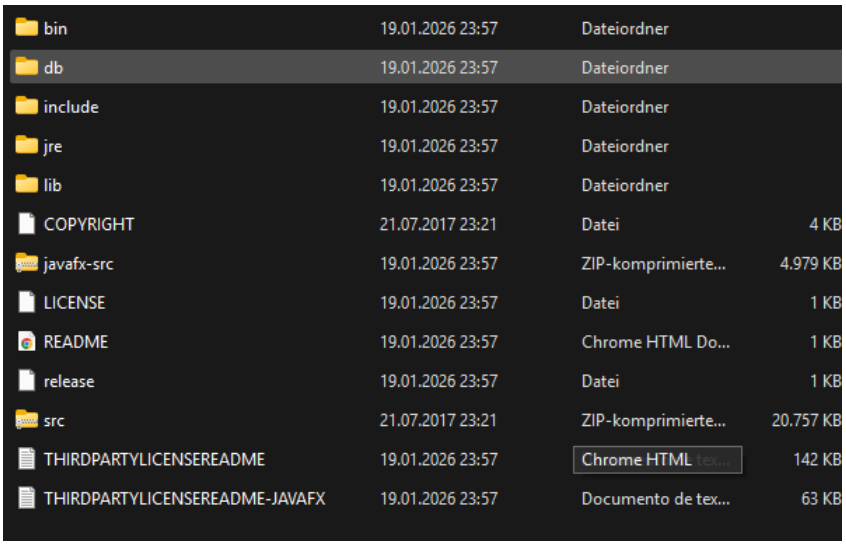
Mapea las tablas utilizando Hibernate con NetBeans y realiza un proyecto Java llamado HibernateTarea4 que realice lo siguiente:

1. Crea la base de datos y realiza el proyecto Hibernate con las clases generadas.
2. Realiza una inserción, un borrado y un update sobre la tabla empleados.
3. Obtener un listado sobre las Clases necesarias para visualizar **emp_no, apellido, oficio, salario, dnombre y loc**.
4. Usando load() visualiza los empleados de un departamento.

Lo primero que vamos a hacer es instalar la versión de java con Hibernate (1.8) y la versión de Netbeans (8.2) que nos dan los recursos de la unidad.

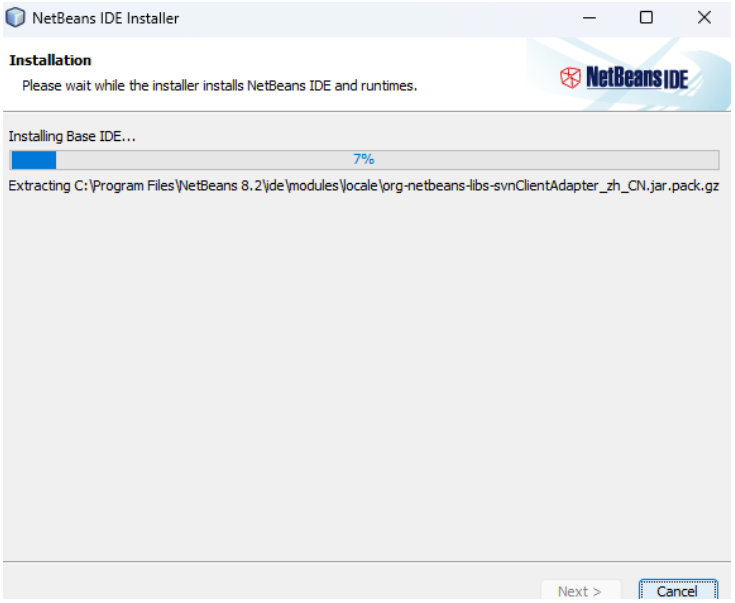
Tenía antes instalada java Zulu 23 y Netbeans 23, pero creo que si los instalo en distintas carpetas no habrá problema de compatibilidad.

1) Instalamos Java primero, no nos ha dado ningún problema. Solo hemos tenido que aceptar a todo



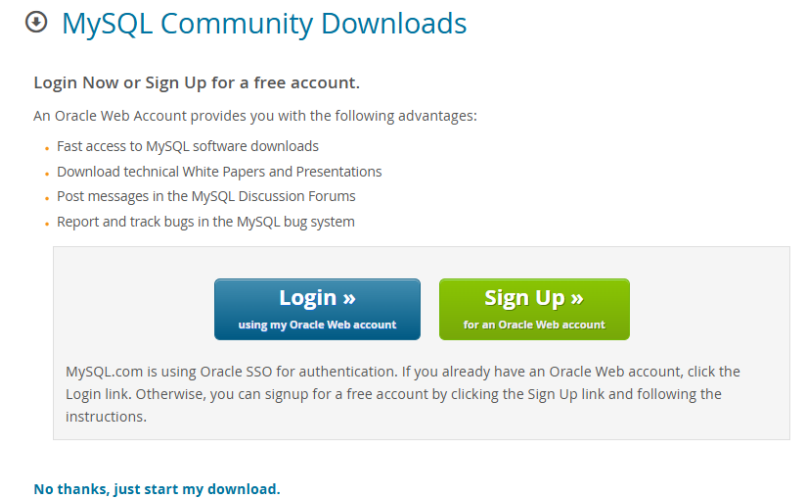
bin	19.01.2026 23:57	Dateiordner	
db	19.01.2026 23:57	Dateiordner	
include	19.01.2026 23:57	Dateiordner	
jre	19.01.2026 23:57	Dateiordner	
lib	19.01.2026 23:57	Dateiordner	
COPYRIGHT	21.07.2017 23:21	Datei	4 KB
javafx-src	19.01.2026 23:57	ZIP-komprimierte...	4.979 KB
LICENSE	19.01.2026 23:57	Datei	1 KB
README	19.01.2026 23:57	Chrome HTML Do...	1 KB
release	19.01.2026 23:57	Datei	1 KB
src	21.07.2017 23:21	ZIP-komprimierte...	20.757 KB
THIRDPARTYLICENSEREADME	19.01.2026 23:57	Chrome HTML	142 KB
THIRDPARTYLICENSEREADME-JAVAFX	19.01.2026 23:57	Documento de tex...	63 KB

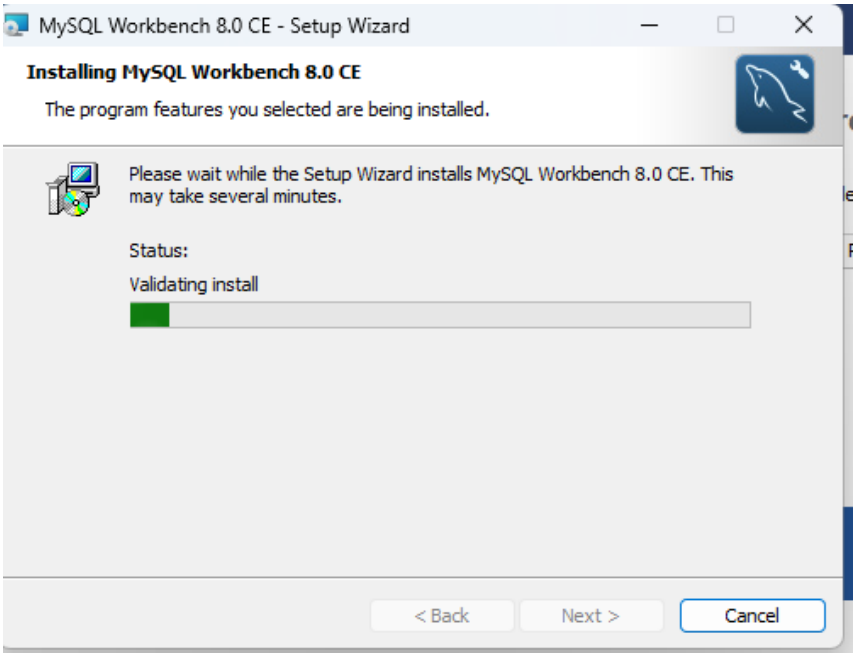
2) Después Netbeans. Por suerte, ha encontrado la versión que queríamos de java a la primera. Le damos que sí a todo salvo a buscar actualizaciones.



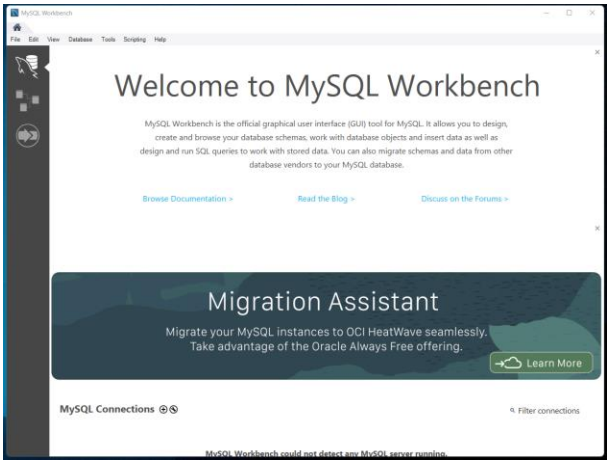
3) Introducción de los datos en la base de datos con MySQL Workbench

Descargamos e instalamos desde la página oficial

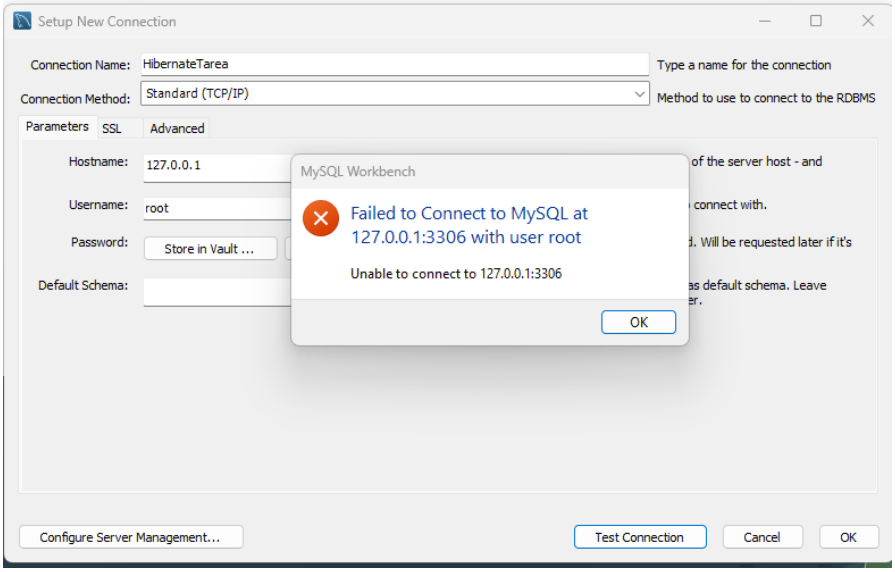




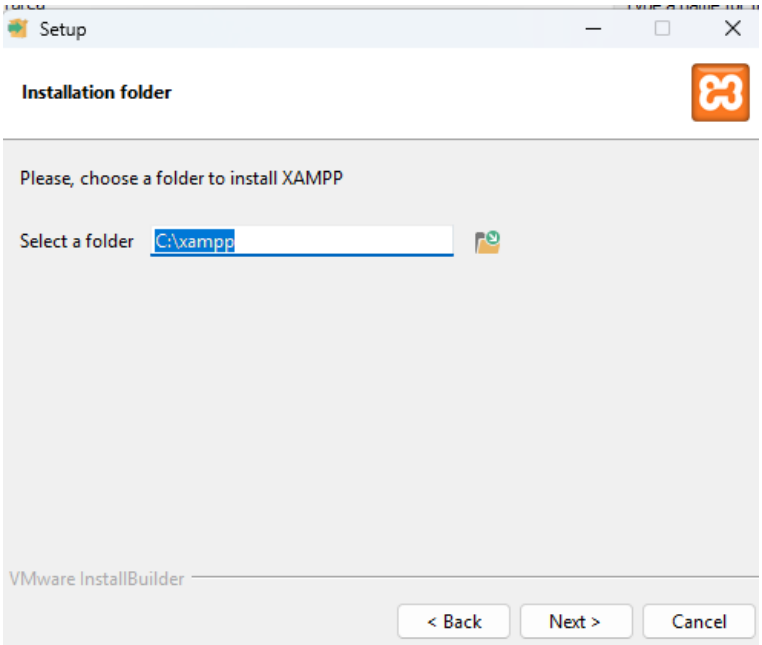
Abrimos MySQL



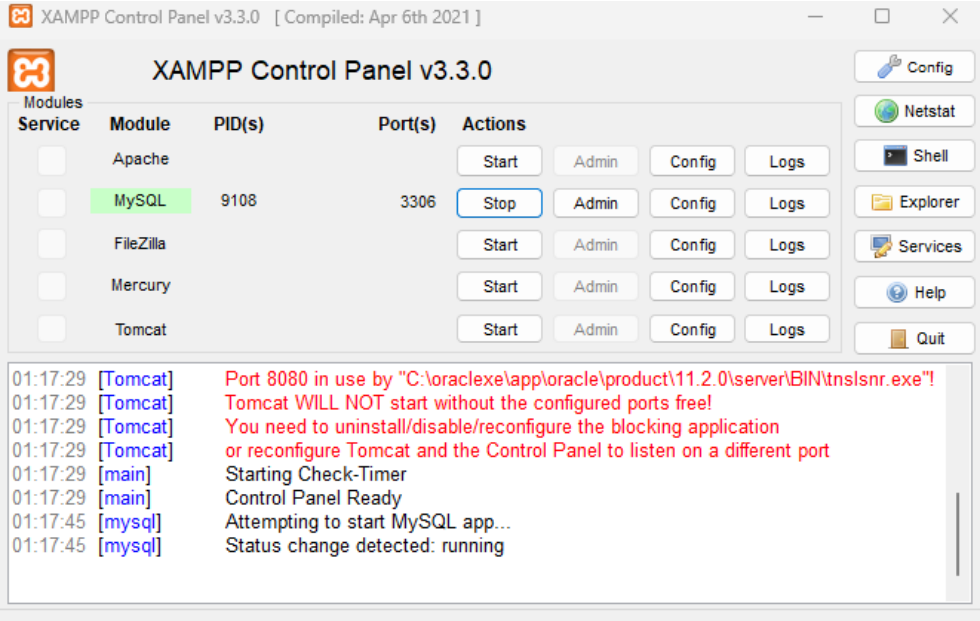
No nos deja crear una conexión, supongo que porque no habrá servidor



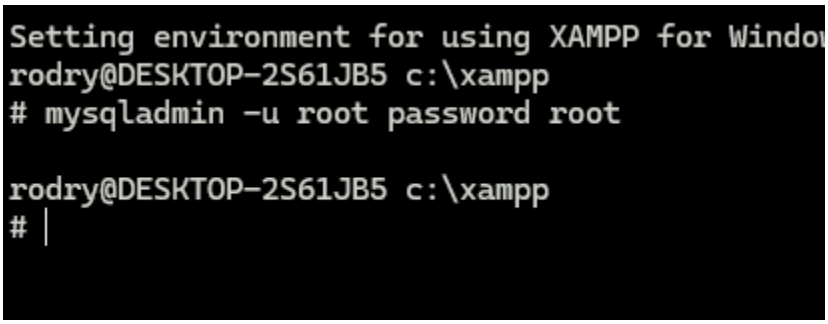
Instalamos XAMPP desde la web oficial



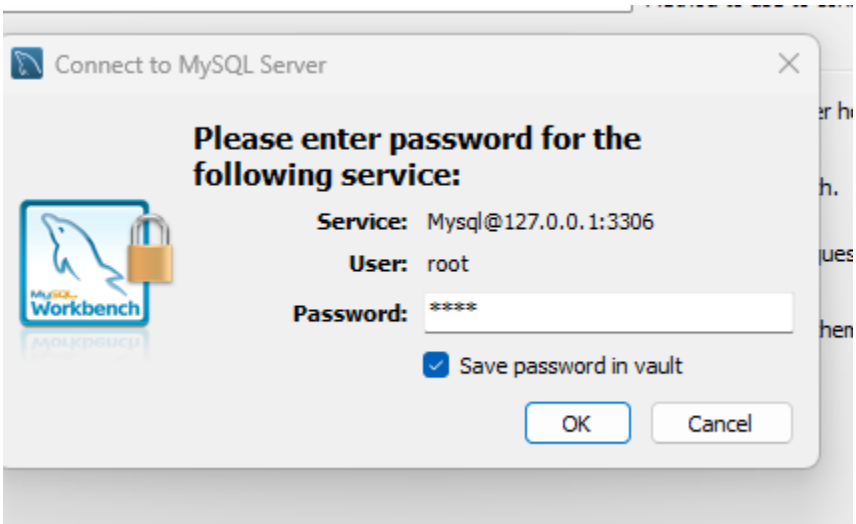
Una vez instalado, abro el panel e inicio el servicio MySQL.

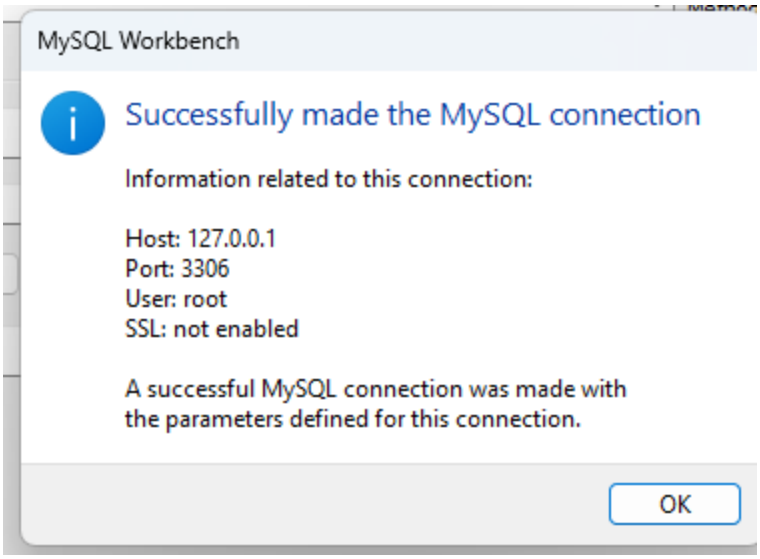


Y configuro la contraseña que pide la tarea.

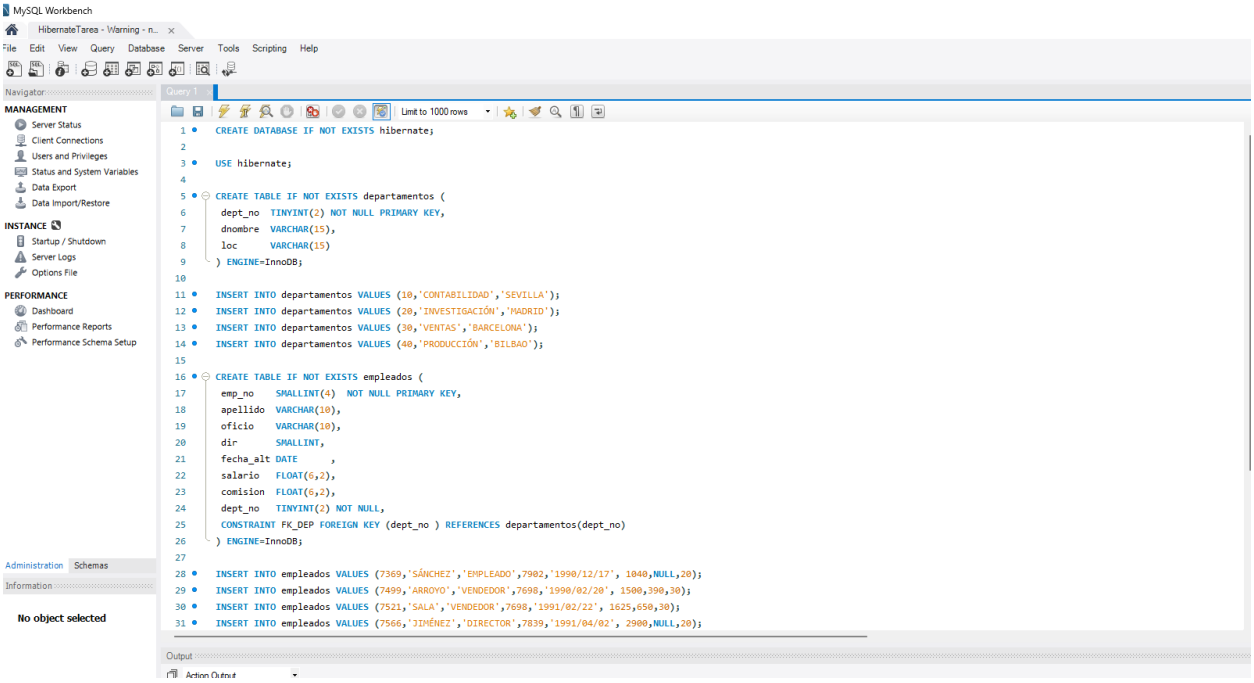


Y ahora, en MYSQL WORKBENCH, funciona la conexión:

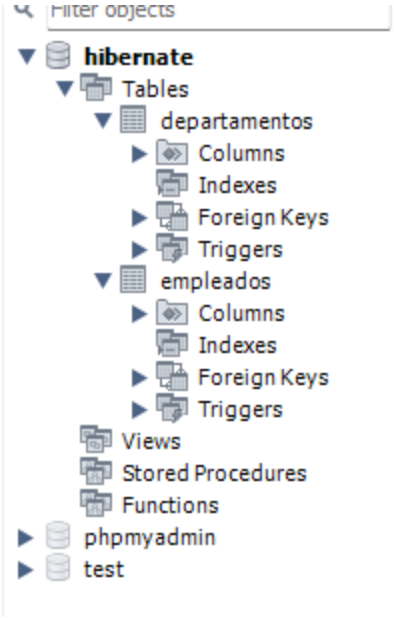


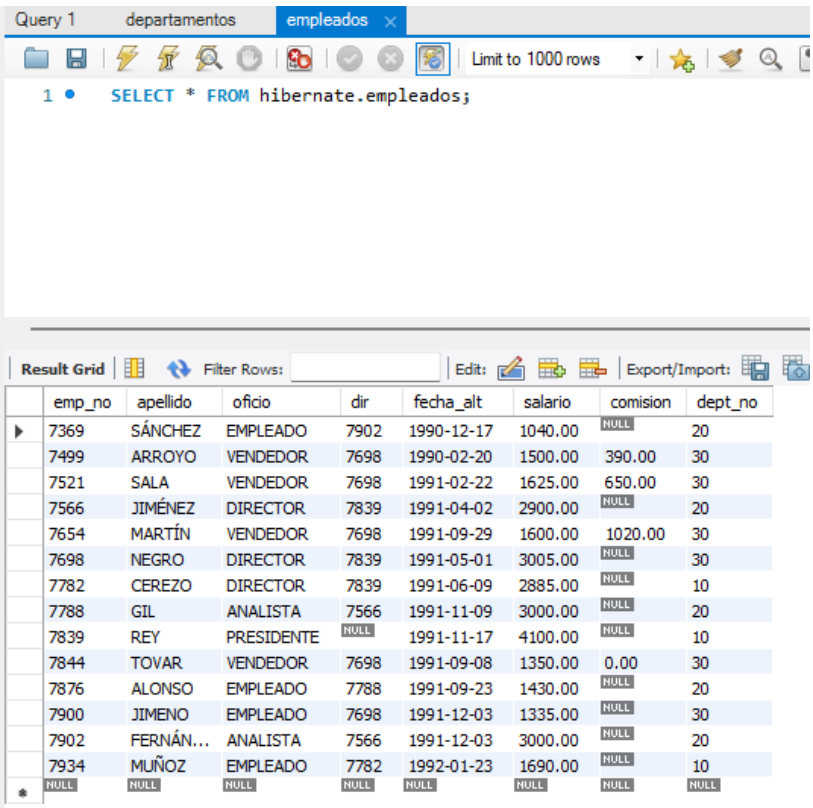


Ya podemos crear la BBDD e introducir los datos de la tarea.

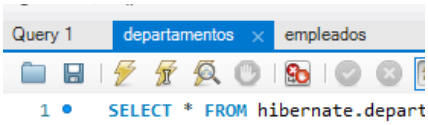


#	Time	Action	Message	Duration / Fetch
18	01:28:21	INSERT INTO empleados VALUES (7844,'TOVAR','VENDEDOR',7698,'1991/09/08', 1350,0,30)	1 row(s) affected	0,000 sec
19	01:28:21	INSERT INTO empleados VALUES (7876,'ALONSO','EMPLEADO',7788,'1991/09/23', 1430,NULL,20)	1 row(s) affected	0,000 sec
20	01:28:21	INSERT INTO empleados VALUES (7900,'JIMENO','EMPLEADO',7698,'1991/12/03', 1335,NULL,30)	1 row(s) affected	0,000 sec
21	01:28:21	INSERT INTO empleados VALUES (7902,'FERNÁNDEZ','ANALISTA',7566,'1991/12/03', 3000,NULL,20)	1 row(s) affected	0,016 sec
22	01:28:21	INSERT INTO empleados VALUES (7934,'MUÑOZ','EMPLEADO',7782,'1992/01/23', 1690,NULL,10)	1 row(s) affected	0,000 sec
23	01:28:21	COMMIT	0 row(s) affected	0,000 sec



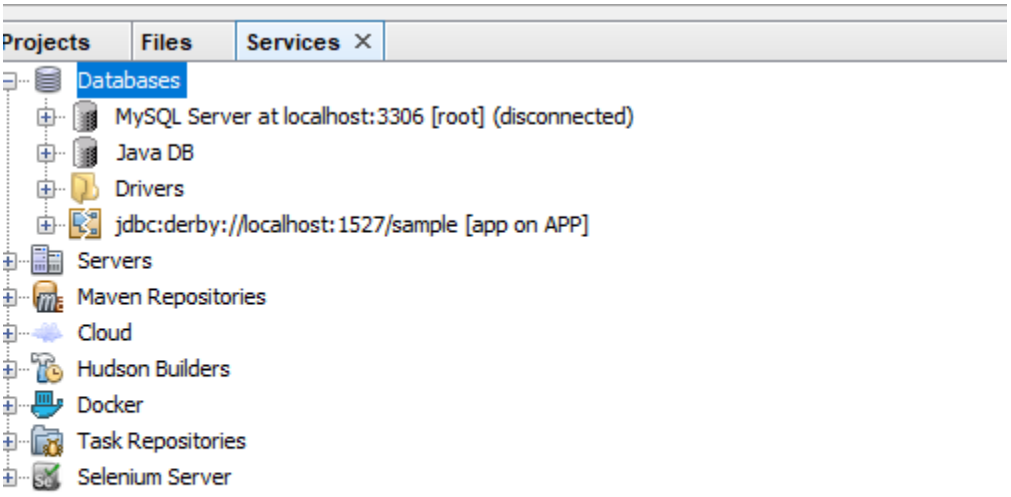


emp_no	apellido	oficio	dir	fecha_alt	salario	comision	dept_no
7369	SÁNCHEZ	EMPLEADO	7902	1990-12-17	1040.00	NULL	20
7499	ARROYO	VENDEDOR	7698	1990-02-20	1500.00	390.00	30
7521	SALA	VENDEDOR	7698	1991-02-22	1625.00	650.00	30
7566	JIMÉNEZ	DIRECTOR	7839	1991-04-02	2900.00	NULL	20
7654	MARTÍN	VENDEDOR	7698	1991-09-29	1600.00	1020.00	30
7698	NEGRO	DIRECTOR	7839	1991-05-01	3005.00	NULL	30
7782	CEREZO	DIRECTOR	7839	1991-06-09	2885.00	NULL	10
7788	GIL	ANALISTA	7566	1991-11-09	3000.00	NULL	20
7839	REY	PRESIDENTE	NULL	1991-11-17	4100.00	NULL	10
7844	TOVAR	VENDEDOR	7698	1991-09-08	1350.00	0.00	30
7876	ALONSO	EMPLEADO	7788	1991-09-23	1430.00	NULL	20
7900	JIMENO	EMPLEADO	7698	1991-12-03	1335.00	NULL	30
7902	FERNÁN...	ANALISTA	7566	1991-12-03	3000.00	NULL	20
7934	MUÑOZ	EMPLEADO	7782	1992-01-23	1690.00	NULL	10
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

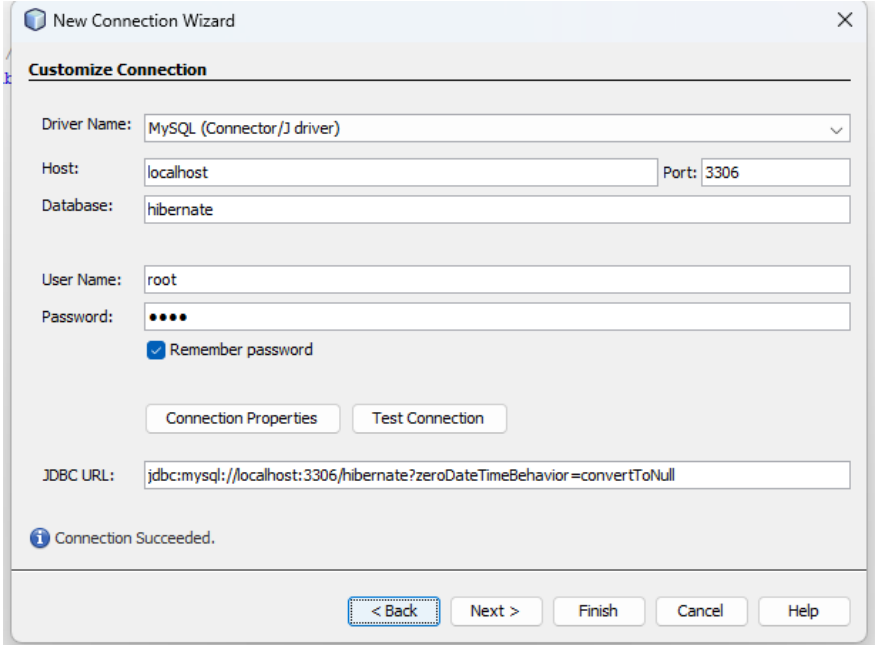


Creamos el proyecto de Java en Netbeans llamado HibernateTarea4.

Conectamos Netbeans y MySQL de esta manera: Abrimos Services >Databases>New Connection...

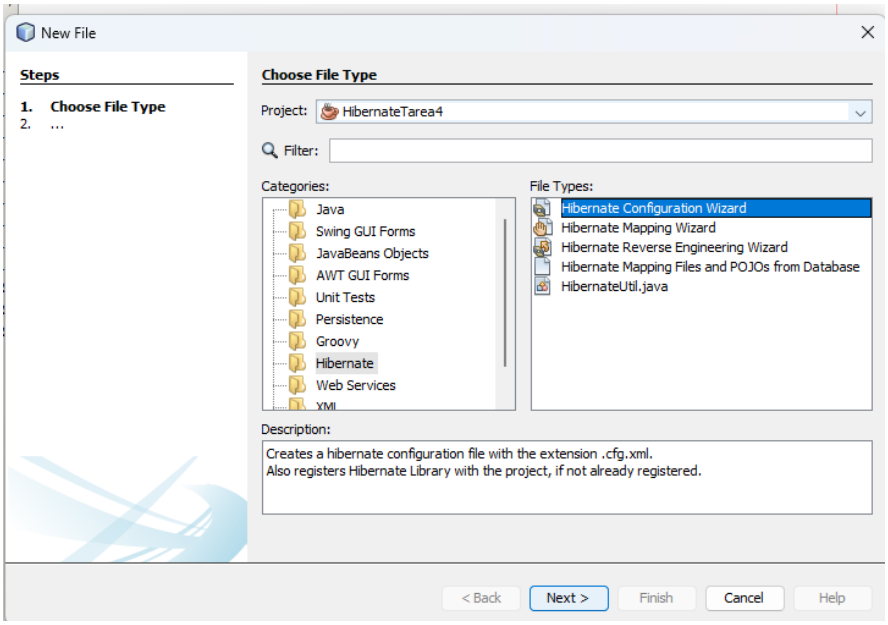


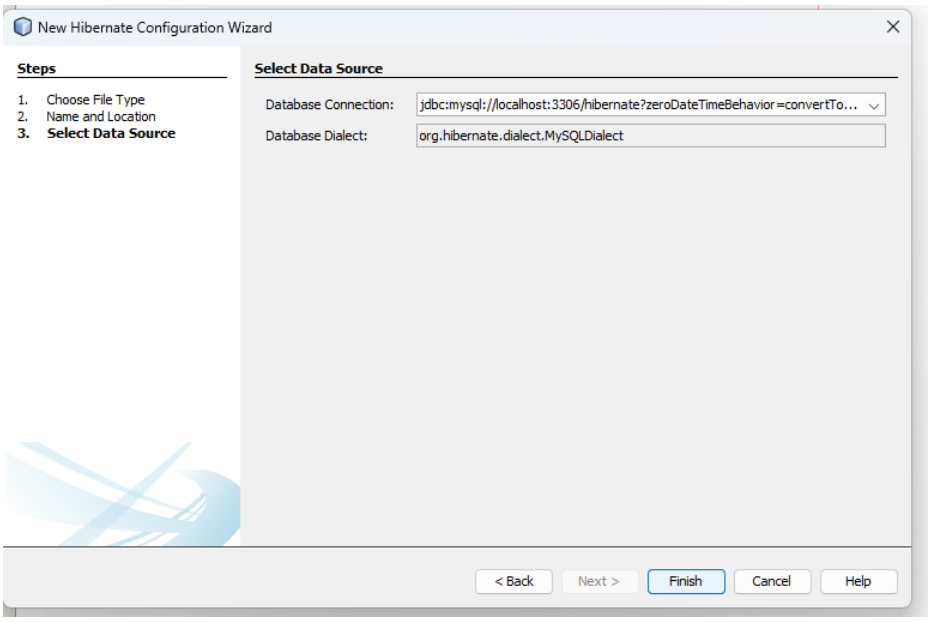
Seleccionamos el Driver de MySQL>Next>Configuramos la conexión



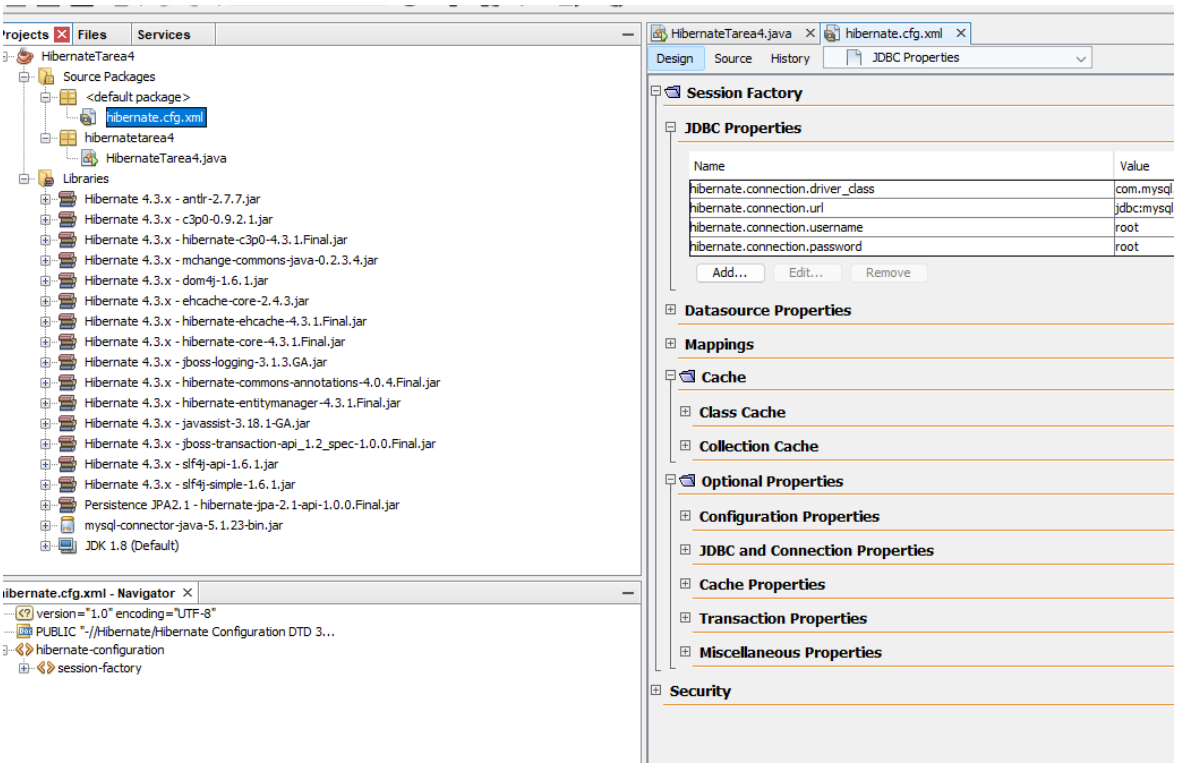
Todo ha ido bien. Pasamos a añadir Hibernate.

Vamos a Projects >Click derecho en HibernateTarea4 > New >Other



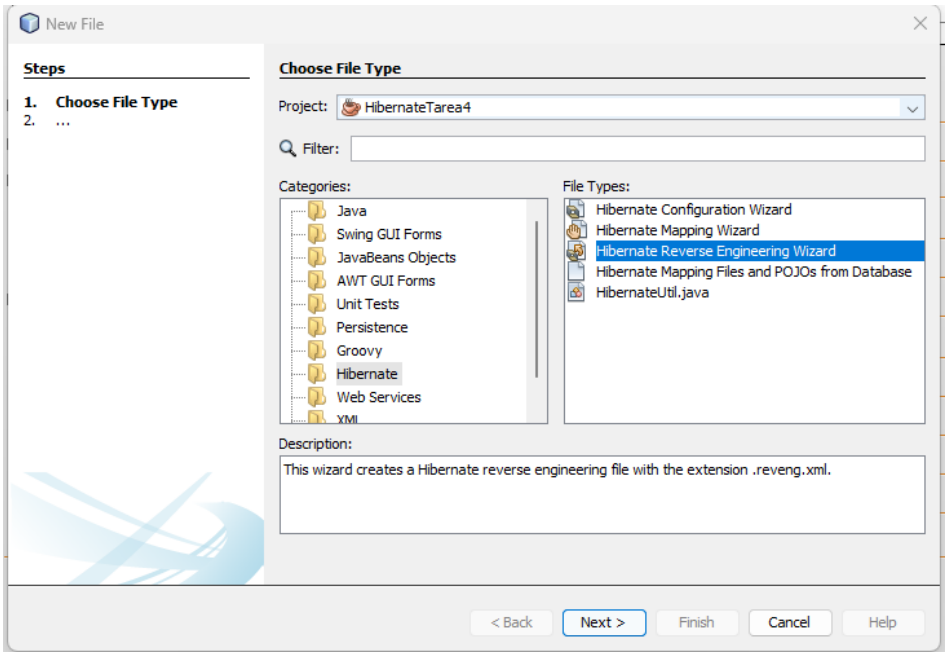


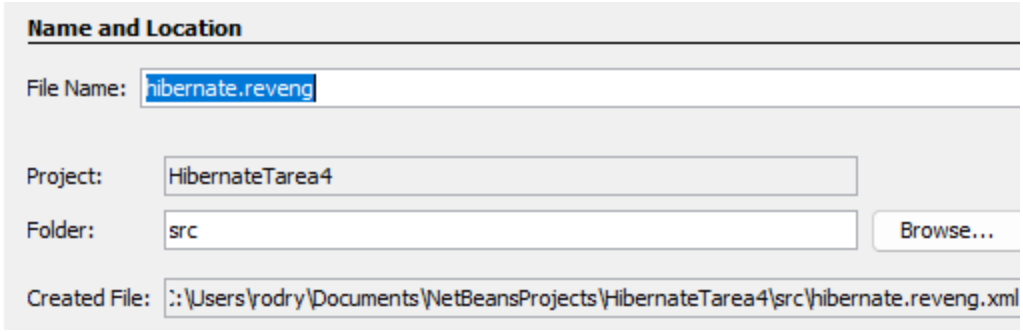
Y se genera el archivo hibernate.cfg.xml, así como se añaden las librerías de hibernate.

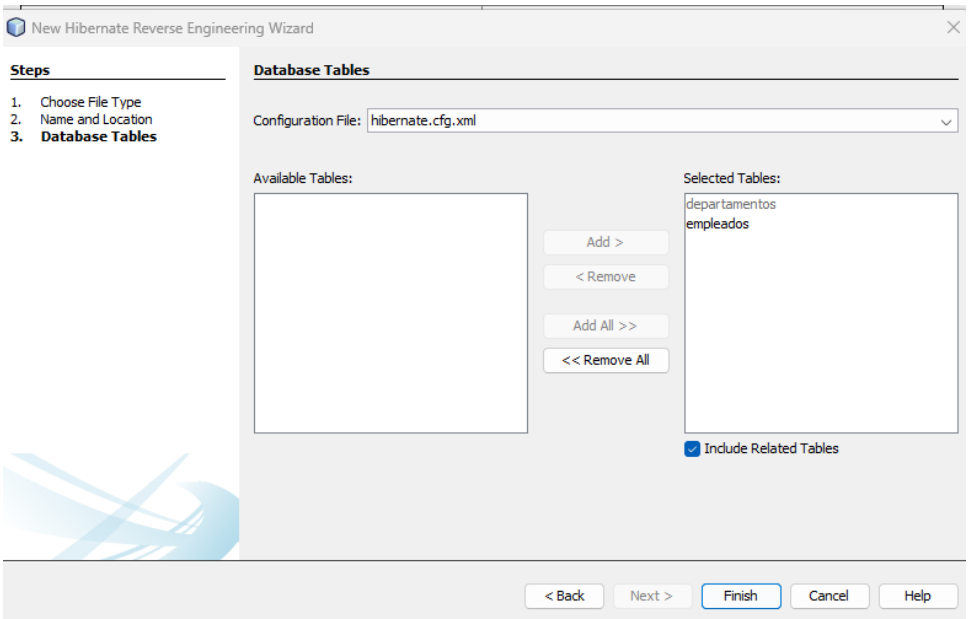


Pasamos a hacer la Ingeniería Inversa:

Click derecho en Source Packages > New>Other >Categories:Hibernate >File Types: Hibernate Reverse Engineering Wizard

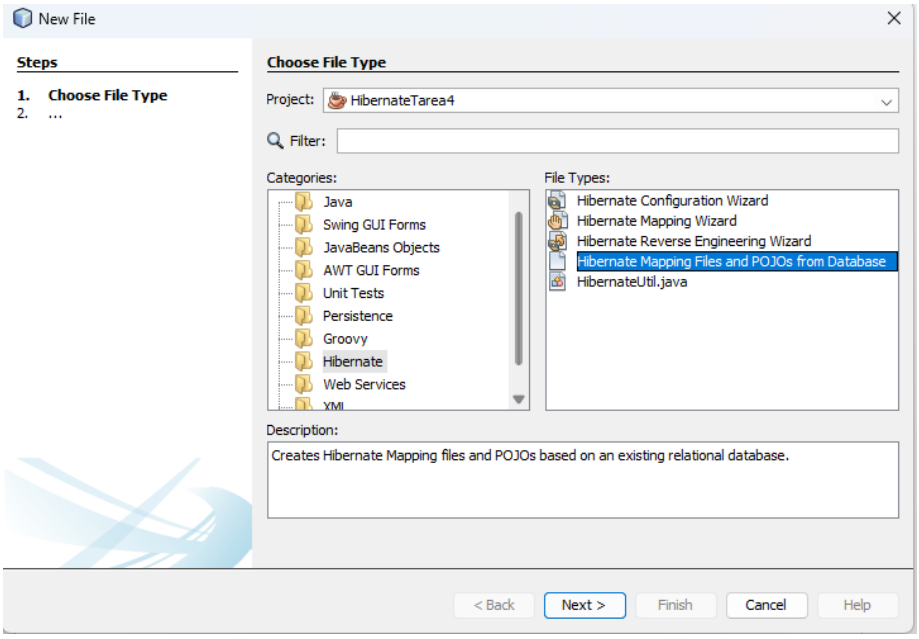


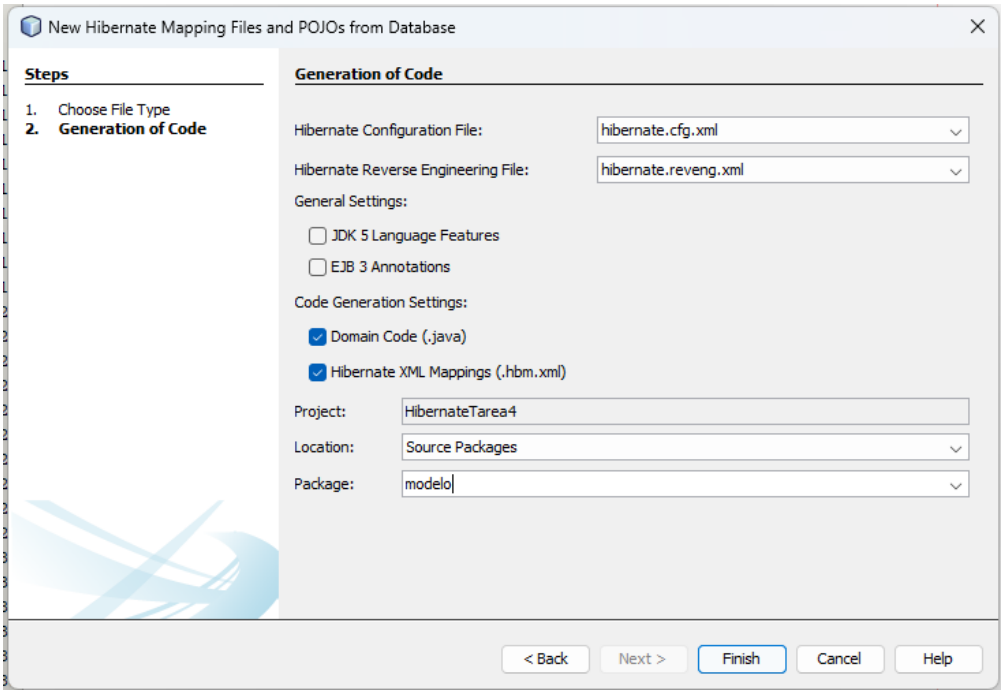




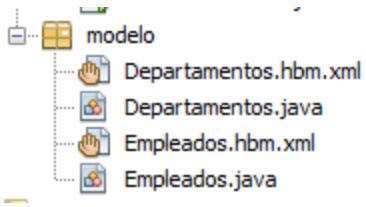
Y se genera el hibernate.reveng.xml

Pasamos a Generar las Clases (POJOs) y el Util

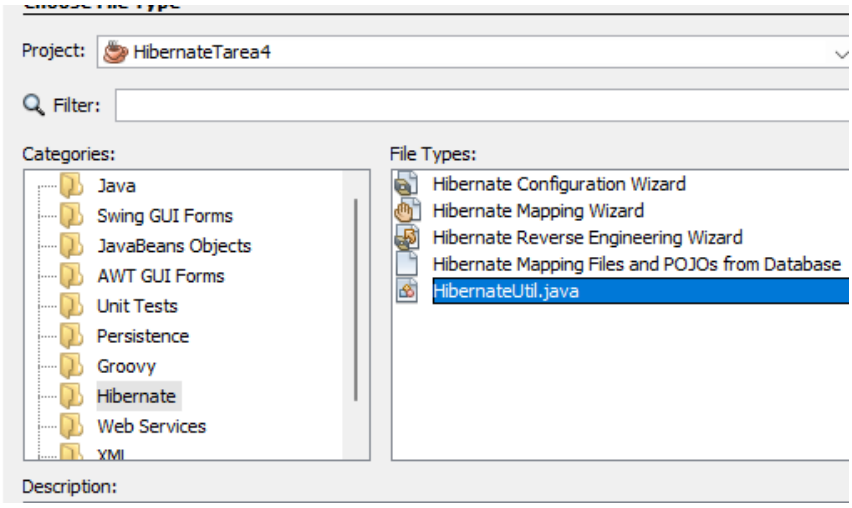


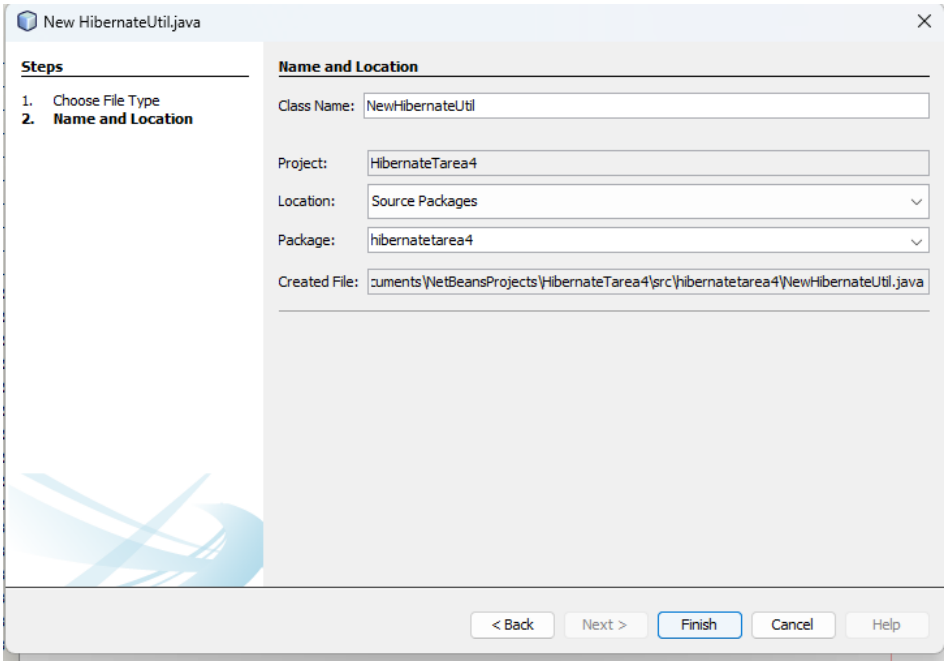


Y se genera



Ahora el HibernateUtil





Y se genera NewHibernateUtil.java

Con esto, el punto 1 ya estaría completo:

1. Crea la base de datos y realiza el proyecto Hibernate con las clases generadas.

Los puntos 2,3 y 4 piden:

2. Realiza una inserción, un borrado y un update sobre la tabla empleados.
3. Obtener un listado sobre las Clases necesarias para visualizar **emp_no, apellido, oficio, salario, dnombre y loc**.

4. Usando load() visualiza los empleados de un departamento.

Todos los apartados son código, lo podemos escribir todo en el main.

```
package hibernatetarea4;

import java.util.Date;
import java.util.List;
import java.util.Set;
import modelo.Departamentos;
import modelo.Empleados;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

public class HibernateTarea4 {

    public static void main(String[] args) {

        // Obtener la sesión
        SessionFactory sf = NewHibernateUtil.getSessionFactory();
        Session session = sf.openSession();

        // APARTADO 2: INSERCIÓN, UPDATE Y BORRADO
        System.out.println("INICIO DE CRUD");
        Transaction tx = session.beginTransaction();

        try {
```

```
            // A) INSERTAR: Necesitamos un Departamento existente
(como por ej: 10) para asignarlo

            // Usamos (byte) porque en la BDD es TINYINT

            Departamentos dep10 = (Departamentos)
session.get(Departamentos.class, (byte)10);


            // Creamos el empleado nuevo

            Empleados nuevoEmp = new Empleados();

            nuevoEmp.setEmpNo((short) 9999); // ID manual (SMALLINT
requiere short)

            nuevoEmp.setApellido("TEST_USER");

            nuevoEmp.setOficio("BECARIO");

            nuevoEmp.setFechaAlt(new Date());

            nuevoEmp.setSalario(1000.0f);

            nuevoEmp.setDepartamentos(dep10); // Asignamos la relación
(Foreign Key)


            session.save(nuevoEmp);

            System.out.println("-> Insertado empleado: " +
nuevoEmp.getApellido());


            // B) UPDATE: Modificar el salario

            nuevoEmp.setSalario(5000.0f);

            session.update(nuevoEmp);

            System.out.println("-> Actualizado salario a: " +
nuevoEmp.getSalario());


            // C) DELETE: Borrar el empleado

            session.delete(nuevoEmp);
```

```
            System.out.println("-> Borrado empleado 9999.");

            // Confirmar cambios en la base de datos. No hace nada
hasta esta orden.
            tx.commit();

        } catch (Exception e) {
            if (tx != null) tx.rollback(); // Si falla, deshacer
            System.err.println("Error en la transacción: " +
e.getMessage());
        }

        System.out.println("\n");

        // APARTADO 3: Visualizar: emp_no, apellido, oficio, salario,
dnombre y loc
        System.out.println("LISTADO ESPECIFICO (HQL JOIN) ");

        // Usamos los nombres de las CLASES y ATRIBUTOS de Java, no de
SQL
        String hql = "SELECT e.empNo, e.apellido, e.oficio, e.salario,
d.dnombre, d.loc " +
                     "FROM Empleados e JOIN e.departamentos d";

        Query query = session.createQuery(hql);
        List<Object[]> resultados = query.list();

        // Recorremos el resultado(lista de arrays de objetos)
```

```
        for (Object[] fila : resultados) {
            System.out.println("Emp: " + fila[0] +
                               " | Apellido: " + fila[1] +
                               " | Oficio: " + fila[2] +
                               " | Salario: " + fila[3] +
                               " | Dept: " + fila[4] +
                               " (" + fila[5] + ")");
        }
        System.out.println("\n");

        // APARTADO 4: USAR LOAD() PARA VER EMPLEADOS DE UN
DEPARTAMENTO
        System.out.println("CARGA DE EMPLEADOS POR DEPARTAMENTO CON
LOAD()");

        // Usamos load para cargar el departamento 20 (Investigación)
en vez de get()
        // Usamos lazy load, asume que el objeto existe sin consultar
la BDD inmediatamente
        try {
            Departamentos depto = (Departamentos)
session.load(Departamentos.class, (byte)20);

            System.out.println("Departamento cargado: " +
depto.getDnombre());
            System.out.println("Lista de empleados:");

            // empleadoses es el nombre por defecto que pone NetBeans
a un Set
```

```
            // Al tocar la colección, Hibernate hace la consulta real
para traer los empleados
            Set<Empleados> lista = depto.getEmpleadoses();


            for (Empleados emp : lista) {
                System.out.println(" - " + emp.getApellido() + " (" +
emp.getOficio() + ")");
            }


        } catch (Exception e) {
            System.err.println("Error al cargar departamento: " +
e.getMessage());
        }


        // Cerramos la sesión al terminar
        session.close();
        System.exit(0);
    }
}
```

Y nos da como resultado:

APARTADO 2

```
INFO: HHH000397: Using ASTQueryTranslatorFactory
INICIO DE CRUD
-> Insertado empleado: TEST_USER
-> Actualizado salario a: 5000.0
-> Borrado empleado 9999.
```

APARTADO 3

```
LISTADO ESPECIFICO (HQL JOIN)
Emp: 7369 | Apellido: SÁNCHEZ | Oficio: EMPLEADO | Salario: 1040.0 | Dept: INVESTIGACIÓN (MADRID)
Emp: 7499 | Apellido: ARROYO | Oficio: VENDEDOR | Salario: 1500.0 | Dept: VENTAS (BARCELONA)
Emp: 7521 | Apellido: SALA | Oficio: VENDEDOR | Salario: 1625.0 | Dept: VENTAS (BARCELONA)
Emp: 7566 | Apellido: JIMÉNEZ | Oficio: DIRECTOR | Salario: 2900.0 | Dept: INVESTIGACIÓN (MADRID)
Emp: 7654 | Apellido: MARTÍN | Oficio: VENDEDOR | Salario: 1600.0 | Dept: VENTAS (BARCELONA)
Emp: 7698 | Apellido: NEGRO | Oficio: DIRECTOR | Salario: 3005.0 | Dept: VENTAS (BARCELONA)
Emp: 7782 | Apellido: CEREZO | Oficio: DIRECTOR | Salario: 2885.0 | Dept: CONTABILIDAD (SEVILLA)
Emp: 7788 | Apellido: GIL | Oficio: ANALISTA | Salario: 3000.0 | Dept: INVESTIGACIÓN (MADRID)
Emp: 7839 | Apellido: REY | Oficio: PRESIDENTE | Salario: 4100.0 | Dept: CONTABILIDAD (SEVILLA)
Emp: 7844 | Apellido: TOVAR | Oficio: VENDEDOR | Salario: 1350.0 | Dept: VENTAS (BARCELONA)
Emp: 7876 | Apellido: ALONSO | Oficio: EMPLEADO | Salario: 1430.0 | Dept: INVESTIGACIÓN (MADRID)
Emp: 7900 | Apellido: JIMENO | Oficio: EMPLEADO | Salario: 1335.0 | Dept: VENTAS (BARCELONA)
Emp: 7902 | Apellido: FERNÁNDEZ | Oficio: ANALISTA | Salario: 3000.0 | Dept: INVESTIGACIÓN (MADRID)
Emp: 7934 | Apellido: MUÑOZ | Oficio: EMPLEADO | Salario: 1690.0 | Dept: CONTABILIDAD (SEVILLA)
```

APARTADO 4

```
CARGA DE EMPLEADOS POR DEPARTAMENTO CON LOAD()
Departamento cargado: INVESTIGACIÓN
Lista de empleados:
 - FERNÁNDEZ (ANALISTA)
 - JIMÉNEZ (DIRECTOR)
 - ALONSO (EMPLEADO)
 - GIL (ANALISTA)
 - SÁNCHEZ (EMPLEADO)
BUILD SUCCESSFUL (total time: 1 second)
```